RAZORPAY BUILDATHON · TRACK 01 · AI GROWTH & AGENTIC COMMERCE

AEGIS RAIL

A Universal Trust & Translation Gateway That Makes Every Razorpay Merchant Safely Transactable by Any AI Buyer Agent

PROJECT PROPOSAL & TECHNICAL BLUEPRINT

Prepared for the Razorpay Buildathon Track 01 submission
Built on Razorpay Test-Mode APIs, the Razorpay MCP Server, and the Claude Agent SDK
Chennai Institute of Technology · AD4V38 — AI in Cyber Security

0. How to Read This Document

This document is written so that someone who has never seen the project before — a judge, a mentor, a teammate, or Razorpay's own team — can read it start to finish and understand not just what AEGIS RAIL *is*, but why it needs to exist, how it actually works end to end, and where its honest limits are. It is organized as a story first, then an architecture, then a hard look at what could go wrong and what we are choosing not to promise.

Two characters recur throughout this document to keep the ideas grounded in something real:

- **Meera** — runs a small handmade-candle and home-fragrance business. She accepts payments through a basic Razorpay Payment Page. She has never heard of ACP, AP2, or agentic commerce.
- **Nova** — a fictional AI shopping assistant, the kind that will soon live inside browsers, chat apps, and phones, capable of discovering products and completing purchases on a user's behalf without a human clicking "buy."

Everything AEGIS RAIL does can be summarized as: **making sure Meera can sell to Nova, safely, without Meera having to become a protocol engineer, and without Razorpay having to trust Nova blindly.**

1. The Problem — Told as a Story

> *It is early 2027. Nova, an AI shopping assistant living inside a popular chat app, has just been asked by its user:* **"find me a soy candle under ₹600 that smells like sandalwood, and just buy it if it's good."**
>
> *Nova searches. It finds Meera's shop. Meera's product photos are lovely, her prices are fair, her reviews are excellent. But Nova cannot read her catalog — there is no machine-readable price, stock, or policy feed. Nova cannot verify her checkout speaks any protocol it understands — no ACP, no AP2, no UCP. And even if it could reach her Razorpay Payment Page, there is no way for Meera's side to verify that "Nova" is really an authorized, spending-limited agent acting for a real human, and not a scripted bot probing for stolen-card testing or checkout abuse.*
>
> *Nova gives up on Meera and buys from a large, already-agent-ready competitor instead. Meera never even knows she lost the sale.*

This single scene is the entire Track 01 brief compressed into one moment: "*Build an agent that grows revenue for a merchant on Razorpay test-mode APIs, or that makes a merchant transactable by an AI buyer end to end.*" Our research (detailed in Section 2) shows this exact scene — a small, honest merchant being invisible to AI buyers — is the one gap that none of Razorpay's current agentic products, and none of the global protocol standards, have actually closed yet.

2. What Razorpay Has Already Solved (So We Don't Rebuild It)

Before proposing anything, we mapped what Razorpay has already shipped in production, so AEGIS RAIL is additive to their roadmap rather than a duplicate of it.

Already shipped by Razorpay	What it does	Why we are not rebuilding this
Agent Studio (Claude Agent SDK)	Marketplace of production-ready AI agents for merchants	This is Razorpay's own platform; competing with it head-on is redundant
Abandoned Cart Conversion Agent	Voice-led cart recovery, built with Nugget/Zomato and SuperU	Already solved, live in pilot
Dispute Responder Agent	Automated chargeback evidence submission	Already solved, live in pilot
Subscription Recovery Agent	Voice-based failed-subscription recovery with ElevenLabs	Already solved, live in pilot
Cashflow Forecaster Agent	3–7 day cash position and shortfall prediction	Already solved, live in pilot
In-app conversational checkout pilots	Zomato, Swiggy, Zepto, PVR Inox, Vodafone Idea, Bluestone, Honasa — via UPI Reserve Pay + NPCI	Enterprise-only, not generalized — this is exactly our whitespace (see Section 3)
Razorpay MCP Server + llms.txt	Lets AI tools call Razorpay APIs directly and read documentation	We build *on top of* this, as our execution layer

The pattern is clear: Razorpay has proven the concept of agentic commerce works, but only for its largest, most resourced merchant partners. Nothing exists yet for the long tail.

3. The Whitespace We Are Targeting

Four gaps came out of the research, and AEGIS RAIL is built to close all four at once, because they are really one problem seen from different angles:

Gap	Evidence
The long tail is invisible to AI buyers. Every named agentic-commerce pilot is a large enterprise brand.	Zomato, Swiggy, Zepto, PVR Inox, Vodafone Idea, Bluestone, Honasa — no small D2C or SMB merchant is mentioned anywhere
No agent-readable catalog standard exists for ordinary merchants. Track 01's own slide lists "agent-readable catalog" as an open example direction, not a solved problem.	Razorpay Buildathon track brief itself
The protocol layer is fragmented and still being fought over. ACP (Stripe/OpenAI), UCP (Google/Shopify), AP2 (Google, with Mastercard/Visa/PayPal), and x402 (Coinbase) all exist simultaneously, plus India's own NPCI Unified Agent Protocol (UAP), which is still only "reportedly" in development.	Multiple 2026 industry analyses describe this as an active, unsettled "protocol race," not a finished stack
Agent identity and fraud verification is the industry's admitted weak point. Multiple sources single out the checkout/authorization moment as where agent-fraud risk concentrates most.	Industry protocol-comparison research, 2026

> **The insight:** nobody needs another single-purpose commerce agent. What's missing is the **connective and protective layer underneath** — something that (a) makes any merchant instantly agent-readable, (b) can speak whichever protocol the AI buyer happens to use, and (c) refuses to move money until it can explain, in plain language, why it trusts the agent asking for it.

4. Introducing AEGIS RAIL

AEGIS RAIL is a universal agent-commerce gateway and trust layer, built on the Razorpay MCP Server and test-mode APIs, that sits between any AI buyer agent and any Razorpay merchant — no matter how small. It does three things no single Razorpay product currently does together:

1. **Translates** — accepts a purchase request in whatever protocol the AI buyer speaks (ACP, AP2, UCP, or a simulated NPCI UAP mandate), and converts it into native Razorpay operations.
2. **Agentifies** — takes a merchant's existing, unstructured product data and auto-generates a machine-readable, queryable catalog, so a merchant like Meera never has to write a line of schema.
3. **Verifies, explains, and gates** — before any money moves, a zero-trust verification engine checks the buyer agent's identity, signed mandate, and spend limits, scores the request for anomalous behavior, and produces a human-readable explanation for every allow or block decision. Nothing is a black box.

> *Six months later, in the same scenario: Nova asks the same question. This time, AEGIS RAIL has already turned Meera's product sheet into an agent-readable catalog behind the scenes. Nova discovers Meera's candle instantly, sends a signed purchase mandate in AP2 format, AEGIS RAIL verifies the mandate's signature and spend limit, scores the request as low-risk, translates it into a Razorpay Order and Payment Link, and completes the transaction — all in under two seconds, all logged, all explainable. Meera gets an SMS: "Sale completed via AI buyer agent. Trust score: 96/100. View reasoning." She never had to touch a line of code.*

5. Agentic Architecture

AEGIS RAIL is a multi-agent system orchestrated on the Claude Agent SDK — deliberately the same foundation Razorpay itself has already standardized on, which makes the system easy for their engineering team to absorb, extend, or acquire.

5.1 Layered System View

LAYER 1 — PROTOCOL TRANSLATION (the "front door")

Ingests inbound purchase intents from AI buyer agents in ACP, AP2, UCP formats, or a simulated NPCI UAP mandate. Normalizes each into one internal format: the **Canonical Intent Object** (buyer identity, merchant target, cart contents, signed spend mandate, protocol origin, timestamp).

↓

LAYER 2 — CATALOG AGENTIFICATION ENGINE

Ingests a merchant's raw product data (spreadsheet export, Razorpay Payment Page config, or simple product list) and auto-generates a structured, agent-queryable catalog — price, stock, variant, policy, and return-window fields — exposed as MCP tool definitions so any AI buyer agent can query it directly. This is a one-time onboarding step per merchant, run by a dedicated **Catalog Agent**.

↓

LAYER 3 — AGENT IDENTITY & MANDATE VERIFICATION GATEWAY (the core)

This is the security layer, and it is deliberately built zero-trust: every incoming Canonical Intent Object is treated as unverified until proven otherwise, every single time — no standing trust, no session memory.

- **Signature & mandate check** — verifies the cryptographic signature on the buyer's mandate and confirms the requested purchase is within the declared spend limit, scope, and expiry window.
- **Anomaly & fraud scoring** — a lightweight ML classifier (trained on synthetic transaction-behavior data) scores the request for red flags: velocity anomalies, spend-limit creep across repeated calls, catalog-scraping patterns, replayed signatures.
- **Explainability engine** — every allow/block decision is accompanied by a SHAP-style feature-attribution explanation in plain language ("blocked: spend request is 4.2x this agent's typical basket size, and mandate expiry is 40 seconds away"). This directly satisfies Track 01's own stated bar: every money action must be explainable, bounded, and gated.

↓

LAYER 4 — ORCHESTRATOR AGENT

The coordination brain, built on the Claude Agent SDK. Routes verified intents to the right sub-agent (Catalog Agent for discovery queries, Negotiation Agent for upsell/cross-sell suggestions, Payment Execution Agent for the actual transaction), and — critically — handles graceful failure: if an agent checkout attempt fails or times out, the Orchestrator automatically falls back to generating a standard Razorpay Payment Link that a human can complete, so the sale is never fully lost.

↓

LAYER 5 — RAZORPAY EXECUTION LAYER

All actual money movement happens exclusively through the official **Razorpay MCP Server** and test-mode REST APIs — Orders, Payment Links, Payment Pages, and Settlements. AEGIS RAIL never talks to a payment rail directly; it only ever asks Razorpay to do it, which keeps the whole system compliant-by-construction with Razorpay's own trust boundary.

↓

LAYER 6 — AUDIT & EXPLAINABILITY LEDGER

Every decision — allowed or blocked — is written to a hash-chained, tamper-evident log alongside its full reasoning trace. A simple merchant-facing dashboard lets someone like Meera see, in plain English, exactly why every AI-agent transaction touching her shop was approved or refused.

5.2 The Sub-Agent Roster

Agent	Role	Primary Tooling
Catalog Agent	Builds and maintains the agent-readable product feed per merchant	Structured extraction + MCP tool schema generation
Protocol Adapter Agent	Speaks ACP / AP2 / UCP / simulated-UAP and normalizes to the Canonical Intent Object	Protocol-specific parsers, JSON schema validation
Trust & Verification Agent	Signature checks, mandate/spend-limit enforcement, anomaly scoring, explanation generation	Cryptographic verification, gradient-boosted anomaly classifier, SHAP
Negotiation / Upsell Agent	Suggests bounded, policy-safe cross-sells within an already-approved transaction	Rule-gated recommendation logic
Payment Execution Agent	Converts the approved intent into real Razorpay Orders / Payment Links via MCP	Razorpay MCP Server, test-mode API keys
Audit Agent	Writes every decision + reasoning trace to the hash-chained ledger	Append-only log, dashboard renderer

5.3 End-to-End Sequence (Meera & Nova, Walked Through Technically)

1. Nova sends a signed AP2-style purchase mandate for Meera's candle to AEGIS RAIL's public endpoint.
2. The **Protocol Adapter Agent** parses the AP2 mandate into a Canonical Intent Object.

3. The **Trust & Verification Agent** checks the mandate signature, confirms the ₹600 spend is within Nova's declared limit, runs the anomaly scorer (result: low risk), and generates the explanation string.
4. The **Orchestrator Agent** approves the intent and routes it to the **Payment Execution Agent**.
5. The Payment Execution Agent calls the Razorpay MCP Server to create an Order and a Payment Link in test mode, using Meera's Razorpay account.
6. Nova completes payment autonomously against that link, on the user's behalf.
7. The **Audit Agent** writes the full trace — mandate, risk score, explanation, and transaction ID — to the ledger, and Meera receives her plain-English summary.

6. Why This Matches Track 01's Own Bar

The track brief states the bar explicitly: "*Every money action explainable, bounded and gated. Show the audit trail and one failure handled gracefully.*" AEGIS RAIL is designed around that sentence, not added to satisfy it after the fact:

Requirement	How AEGIS RAIL satisfies it
Explainable	Every allow/block decision carries a plain-language, feature-attributed explanation from the Trust & Verification Agent
Bounded	Every transaction is checked against a signed spend mandate before execution; nothing executes outside the declared limit
Gated	Zero-trust design — no standing trust, no cached approvals; every request is re-verified independently
Audit trail	Hash-chained, tamper-evident ledger with a merchant-facing plain-English dashboard
One failure handled gracefully	Automatic fallback to a standard Razorpay Payment Link if agent-side checkout fails or times out, so the sale is never entirely lost

7. Real-World Limitations & How We Address Them

Being honest about constraints is part of building something credible. Here is every major real-world limitation we identified, and the concrete decision we made to work around it rather than pretend it doesn't exist.

Limitation	Why it exists	How AEGIS RAIL handles it
NPCI's Unified Agent Protocol (UAP) is not publicly live	It is still only reportedly in development as of mid-2026	We build a protocol-compliant **simulator** for UAP-style mandates, structured so the Protocol Adapter Agent can swap in the real spec the moment it ships, without a redesign
ACP, AP2, UCP, and x402 are all still evolving standards, not one settled spec	Each is backed by a different company (OpenAI/Stripe, Google/Mastercard, Google/Shopify, Coinbase) and none has "won" yet	The Protocol Adapter Agent is built modularly — one parser per protocol — so supporting a fifth or dropping a failed one is a contained change, not a rewrite
No real fraud-labeled transaction data exists for a student project	Real agent-fraud datasets are proprietary to large PSPs and card networks	We train the anomaly scorer on carefully constructed synthetic transaction-behavior data, and we report precision/recall honestly as provisional, not production-grade
Only test-mode Razorpay APIs are available for the demo	Real settlement requires a live, KYC-verified merchant account and compliance sign-off	The entire demo runs on Razorpay's test-mode Orders/Payment Links/Payment Pages APIs, which is explicitly what the track brief asks for
Legal and regulatory questions around agent-initiated payments (who is liable if an agent overspends, RBI's stance on agentic mandates) are unresolved industry-wide	This is a genuinely open regulatory question, not a software problem	AEGIS RAIL enforces bounded mandates and full auditability so that, whatever the eventual regulatory answer is, the system already produces the evidence trail regulators would need
Solo build, fixed timeline	One person cannot build and polish every layer to production depth in a buildathon window	Layers 1–3 (translation, catalog agentification, trust gateway) are built to full depth as the demo centerpiece; Layers 4–6 are built functionally, not exhaustively, and this priority order is stated openly rather than hidden

8. What We Are Not Able to Solve (Stated Plainly)

This section exists so nobody — including a Razorpay reviewer — discovers these gaps and feels misled. These are the honest edges of the project.

- **No live federation with real ACP/AP2/UCP partner networks.** Those ecosystems (OpenAI's ChatGPT checkout, Google's Shopify integration) are closed or invite-only in 2026. We can build fully protocol-compliant adapters and simulators, but we cannot demonstrate a live handshake with, say, actual ChatGPT checkout infrastructure.
- **No real-money settlement or regulatory certification.** This remains firmly a test-mode demonstration. Turning this into a certifiable production system requires RBI/NPCI compliance work far beyond a buildathon's scope.
- **Fraud-model accuracy is provisional, not proven.** Without access to real agent-fraud transaction history, our precision/recall numbers describe behavior against synthetic data only, and we say so explicitly rather than implying production-grade accuracy.
- **We cannot fully resolve who is liable when an autonomous agent overspends or is compromised.** That is a legal and policy question the industry itself has not answered; our system can only guarantee that the evidence needed to assign liability correctly is available and intact.
- **Catalog agentification quality depends on how well-structured the merchant's original data is.** A merchant with chaotic, inconsistent product listings will get a lower-quality auto-generated catalog than one with clean data — this is a real, unsolved data-quality ceiling, not something the Catalog Agent can fully engineer around.

9. Why This Is the Bet Worth Making

Most Track 01 submissions will likely build one more version of what Razorpay has already shipped — another cart-recovery bot, another conversational checkout demo layered on a single big-brand merchant. AEGIS RAIL is deliberately positioned one layer underneath that entire category: instead of competing with Razorpay's own Agent Studio products, it solves the problem that makes all of them scale — trustworthy, standardized, protocol-agnostic access to the merchants Razorpay actually has millions of, not the six it has piloted.

This is also the version of the project that plays directly to real strengths already proven in prior work: anomaly detection, explainable AI (SHAP/LIME-style reasoning), and zero-trust security architecture — the same foundations behind SENTINEL-AI — applied here to a live commercial problem Razorpay is actively trying to solve, using their own preferred stack (the Claude Agent SDK) and their own official MCP server. That alignment is the strongest version of the pitch: "*we didn't just build a demo — we understood exactly where your platform's actual gap is, and we built the layer that closes it.*"

> **One-line pitch for judges:** AEGIS RAIL doesn't build another agent for Razorpay's biggest merchants — it builds the trust layer that lets Razorpay's smallest merchants sell to AI buyers at all.
